

Pràctica Bloc II

Marc Mateu, Luis Calvet

November 2025



Índex

1	Introducció	4
1.1	Descripció breu del problema	4
1.2	Objectiu de la pràctica	4
1.3	Algorismes implementats	4
1.4	Justificació algorisme no implementat	5
2	Descripció de l'entorn	5
2.1	Com funciona FrozenLake?	5
2.2	Paper de <code>is_slippery=True</code>	6
2.3	Explicació del terreny lliscant	6
3	Metodologia general	7
3.1	Com s'entrenen els agents?	7
3.1.1	Execucions per a resultats fiables	7
3.2	Mesures d'avaluació	8
4	Algorismes implementats	8
4.1	SARSA	8
4.1.1	Decissió de disseny	8
4.1.2	Fórmules	8
4.1.3	Paràmetres estudiats ($\alpha, \gamma, \varepsilon$)	9
4.1.4	Pros i contres	9
4.2	Q-Learning	10
4.2.1	Diferències amb SARSA	10
4.2.2	Estatègia off-policy	10
4.2.3	Paràmetres i decisions	10
4.2.4	Pros i contres	11
4.3	Monte Carlo	11
4.3.1	Episodis complets	11
4.3.2	Problemes en ambients estocàstics	11
4.3.3	Decisions de disseny	12
4.3.4	Pros i contres	12
4.4	Algorisme Genètic	12
4.4.1	Individu	12
4.4.2	Fitness	13
4.4.3	Mutació i crossover	13
4.4.4	Justificació del disseny	13
4.5	Resultats	13
4.5.1	Taules i gràfics	13
4.5.2	Resultats: Agent SARSA	14
4.5.3	Resultats: Agent Q-Learning	14
4.5.4	Resultats: Agent Monte Carlo	15
4.5.5	Resultats: Algorisme Genètic	16
4.5.6	Anàlisi de les Polítiques Resultants	17

4.6	Hiperparàmetres	19
4.7	Comparació global de resultats	21
5	Ús d'intel·ligència artificial generativa	23

1 Introducció

1.1 Descripció breu del problema

El problema abordat en aquesta pràctica és el joc **Frozen Lake**. L'entorn consisteix en un tauler de dimensions $n \times n$ on l'agent ha de navegar per trobar un camí des de la seva posició inicial fins a una casella objectiu.

El mapa presenta caselles perilloses: els forats. Si l'agent trepitja un d'aquests forats durant el recorregut, perd la partida.

La característica principal que defineix la complexitat d'aquest problema és l'estocasticitat del moviment. Atès que el terreny és lliscant, l'agent no sempre es mou en la direcció que ha triat. Per garantir aquesta condició, s'utilitza l'entorn de *Gymnasium* amb el paràmetre obligatori `is_slippery=True`.

1.2 Objectiu de la pràctica

L'objectiu principal d'aquesta pràctica és aplicar els conceptes i tècniques d'IA apresos durant el curs per dissenyar i desenvolupar agents capaços de resoldre el problema plantejat.

Concretament, la tasca se centra en la implementació d'algorismes d'aprenentatge per reforç i algorismes genètics.

Més enllà de la implementació tècnica, la pràctica té com a finalitat realitzar un anàlisi profund de les solucions. Això inclou exposar i justificar les decisions de disseny preses per a cada algorisme, així com estudiar l'efecte que tenen els diferents paràmetres en la solució final.

Finalment, es cerca comparar els rendiments obtinguts entre les diferents tècniques i analitzar les polítiques resultants, tot relacionant el comportament observat amb la teoria vista a classe.

1.3 Algorismes implementats

Per a la resolució d'aquest problema s'han seleccionat i implementat quatre algorismes diferents, cobrint tant tècniques d'aprenentatge per reforç com de computació evolutiva. D'acord amb els requisits de la pràctica, els algorismes desenvolupats són:

- **SARSA (State-Action-Reward-State-Action):** Un mètode d'aprenentatge per reforç *on-policy* basat en Diferència Temporal (TD).
- **Q-Learning:** Un mètode d'aprenentatge per reforç *off-policy*, també basat en TD, que busca aprendre directament la política òptima independentment de les accions exploratòries realitzades.
- **Monte Carlo:** Un mètode basat en l'experiència d'episodis complets, que actualitza els valors dels estats basant-se en la mitjana dels retorns observats al final de cada episodi.

- **Algorisme Genètic:** Una aproximació basada en la computació evolutiva, on una població d'agents evoluciona al llarg de generacions mitjançant processos de selecció, encreuament i mutació per trobar la seqüència d'accions òptima.

1.4 Justificació algorisme no implementat

Entre els algorismes tractats a classe, s'ha decidit descartar la implementació de tècniques basades en **Programació Dinàmica**.

La principal raó d'aquesta decisió és que els mètodes de Programació Dinàmica requereixen un model complet i perfecte de l'entorn (és a dir, conèixer a priori les probabilitats de transició $P(s'|s, a)$ i la funció de recompensa). L'objectiu d'aquesta pràctica és simular un escenari realista on l'agent desconeix les regles físiques del món. Per tant, s'ha prioritzat l'ús d'algorismes *model-free* (com Q-Learning, SARSA i Monte Carlo), els quals obliguen l'agent a aprendre la política òptima exclusivament a través de l'experiència, l'exploració i la interacció directa amb el joc, sense accés a la matriu de transicions subjacent.

Com hem dit, els algorismes de Programació Dinàmica, es fonamenten en l'actualització iterativa dels valors dels estats utilitzant l'equació de Bellman. La fórmula matemàtica que regeix aquestes actualitzacions és la següent:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_k(s')] \quad (1)$$

On:

- $P(s' | s, a)$ és la probabilitat de transició a l'estat s' donat l'estat s i l'acció a .
- $R(s, a, s')$ és la recompensa obtinguda.
- γ és el factor de descompte.

Com es pot observar explícitament a la fórmula, el terme $P(s' | s, a)$ és indispensable per al càlcul. Atès que en aquest problema l'agent no té accés a la matriu de transicions de l'entorn (no sap *a priori* on el portarà el gel lliscant), l'aplicació directa d'aquesta fórmula és inviable sense una fase prèvia d'aprenentatge del model.

2 Descripció de l'entorn

2.1 Com funciona FrozenLake?

FrozenLake és un entorn de simulació discret que representa un llac congelat en forma de graella (*gridworld*) de dimensions $n \times n$. L'objectiu fonamental del joc és que l'agent trobi un **camí segur** des de la seva posició inicial fins a la casella objectiu.

El mapa es compon de quatre tipus de caselles, representades per lletres, cadascuna amb un efecte diferent sobre la partida:

- **S (Start):** Indica la posició inicial on comença l'agent (normalment la casella superior esquerra).
- **H (Hole):** Forat al gel. Aquestes són les caselles a evitar; si l'agent en trepitja una, cau a l'aigua i l'episodi acaba amb una recompensa nul·la (derrota).
- **G (Goal):** La casella objectiu. Arribar aquí significa completar l'episodi amb èxit i rebre la recompensa (victòria).

En cada pas de temps, l'agent ha de triar una acció de moviment entre quatre possibles: 0: Esquerra, 1: Avall, 2: Dreta, 3: Amunt.

2.2 Paper de `is_slippery=True`

El paràmetre `is_slippery=True` és el factor determinant que converteix FrozenLake d'un problema de cerca de camins trivial a un problema complex d'Aprenentatge per Reforç.

Quan aquest paràmetre està activat, l'entorn esdevé **estocàstic**. Això significa que les transicions d'estat no són deterministes, com ja hem comentat a la no implementació de la Programació Dinàmica: l'acció triada per l'agent no garanteix un resultat únic. Si aquest paràmetre fos **False**, l'agent sempre es mouria exactament cap on ha decidit, i el problema es podria resoldre amb algorismes clàssics de cerca (com A* o BFS resolts a la pràctica anterior) sense necessitat d'aprenentatge. L'activació d'aquest mode obliga l'agent a aprendre polítiques robustes que minimitzin el risc, en lloc de simplement memoritzar un camí.

2.3 Explicació del terreny lliscant

El concepte de "terreny lliscant" fa referència a la incertesa en el moviment. A causa del gel, l'agent pot relliscar i moure's en una direcció perpendicular a la desitjada. En la configuració estàndard de FrozenLake, quan l'agent tria una acció (per exemple, moure's cap a la **Dreta**), hi ha una distribució de probabilitats sobre el resultat real:

- **33.3% de probabilitat** de moure's efectivament a la **Dreta**.
- **33.3% de probabilitat** de relliscar i moure's cap a **Dalt** (perpendicular).
- **33.3% de probabilitat** de relliscar i moure's cap a **Baix** (perpendicular).

Aquesta dinàmica implica que l'agent mai té un control total sobre el seu moviment. Per tant, una política òptima en aquest entorn no és necessàriament la ruta més curta, sinó la ruta més segura (aquella que, tot i que l'agent rellisqui, eviti que caigui en un forat).

3 Metodologia general

3.1 Com s'entrenen els agents?

El procés d'entrenament dels agents segueix l'esquema clàssic d'interacció Agent-Entorn definit en el marc de l'Aprenentatge per Reforç. L'entrenament s'organitza en **episodis**. Un episodi comença amb l'agent a la casella de sortida (S) i finalitza quan aquest arriba a l'objectiu (G) o cau en un forat (H).

Durant cada pas de temps t , succeeix el següent cicle:

1. L'agent observa l'estat actual S_t .
2. L'agent selecciona una acció A_t segons la seva política actual (per exemple, ε -greedy).
3. L'entorn executa l'acció i retorna una recompensa R_{t+1} i el nou estat S_{t+1} .
4. L'agent utilitza aquesta informació per actualitzar el seu coneixement (la Q-Table en el cas de Q-Learning/SARSA o el genoma en el cas de l'algorisme Genètic).

Aquest cicle es repeteix fins que es compleix un criteri d'aturada, que en aquesta pràctica s'ha definit com un nombre màxim d'episodis o la convergència de la política.

3.1.1 Execucions per a resultats fiables

Un dels reptes principals d'aquesta pràctica és l'estocasticitat introduïda pel paràmetre `is_slippery=True`. A causa d'això, una única execució de l'entrenament o una única avaluació de l'agent no són representatives; un agent podria tenir sort i arribar a l'objectiu per atzar, o tenir mala sort i relliscar cap a un forat tot i haver pres la decisió correcta.

Per disminuir aquest efecte i obtenir resultats vàlids, s'ha adoptat la següent metodologia:

- **Repetició d'experiments:** Cada algorisme s'ha executat múltiples vegades amb diferents llavors aleatòries.
- **Alteració dels hiperparàmetres** S'han canviat els valors dels diferents hiperparàmetres durant les diverses execucions per observar com es comporta l'agent corresponent a l'algorisme tractat i veure si el seu comportament és fiable i conseqüent als hiperparàmetres.
- **Finestra lliscant:** Per visualitzar millor l'evolució de la recompensa, s'ha aplicat una mitjana mòbil sobre els últims 100 episodis.

3.2 Mesures d'avaluació

Per comparar el rendiment dels diferents algorismes, s'han utilitzat les mètriques suggerides a l'enunciat, que permeten avaluar tant l'eficàcia com l'eficiència de la solució:

- **Taxa d'èxit (Success Rate):** El percentatge d'episodis en els quals l'agent aconsegueix arribar a la casella objectiu. Donada la dificultat del mapa lliscant, aquesta és la mètrica principal.
- **Recompensa acumulada:** La suma de recompenses obtingudes al llarg dels episodis. Com que FrozenLake només dona 1 punt en guanyar i 0 en la resta, aquesta mètrica està directament correlacionada amb la taxa d'èxit.
- **Temps d'execució:** El temps de còmput necessari per completar l'entrenament, útil per comparar el cost computacional entre mètodes simples (SARSA) i evolutius (Genètics).

4 Algorismes implementats

4.1 SARSA

4.1.1 Decisió de disseny

El disseny triat implementa SARSA com un mètode on-policy, on l'agent actualitza la Q-table utilitzant l'acció realment executada segons la política ϵ -greedy. L'elecció de SARSA és adequada en entorns estocàstics com FrozenLake, ja que l'actualització incorpora la pròpia exploració. S'ha implementat un esquema d'exploració ϵ -greedy amb un decaïment gradual (d'1.0 fins a 0.01), així l'agent comença explorant i progressivament passa a explotar el que ha après per obtenir més recompensa. També s'ha definit un màxim de 100 passes per episodi i 15000 episodis totals per garantir prou experiència. La Q-table s'inicialitza a zero i l'actualització segueix exactament la fórmula de SARSA utilitzant l'acció següent seleccionada per la mateixa política.

4.1.2 Fórmules

L'actualització de SARSA segueix la següent fórmula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

on s és l'estat actual, a l'acció presa, r la recompensa rebuda, s' l'estat següent i a' la nova acció seleccionada amb la mateixa política ϵ -greedy. En el codi, el càlcul es fa mitjançant:

$$\begin{aligned} \text{TD target} &= r + \gamma Q(s', a') \\ Q(s, a) &\leftarrow Q(s, a) + \alpha (\text{TD target} - Q(s, a)) \end{aligned}$$

Aquesta actualització és on-policy perquè utilitza l'acció a' que l'agent realment prendrà segons la política d'exploració.

4.1.3 Paràmetres estudiats (α , γ , ε)

Els tres paràmetres principals analitzats són:

- **Taxa d'aprenentatge α :** controla fins a quin punt els valors nous substitueixen els antics. Valors massa grans poden fer el procés inestable, mentre que valors massa petits poden fer que la convergència sigui molt lenta.
- **Factor de descompte γ :** determina la importància de les recompenses futures. En FrozenLake un valor elevat afavoreix trobar camins que conduixin a l'objectiu encara que siguin llargs.
- **Paràmetre d'exploració ε :** regula la probabilitat d'escollir accions aleatòries. S'utilitza un ε decaient des d'1.0 fins a 0.01 amb un factor de reducció constant, permetent una exploració inicial intensa i una explotació final estable.

Els valors utilitzats durant l'entrenament són: 15000 episodis, 100 passes màximes per episodi i un decaïment d'exploració de 0.999.

4.1.4 Pros i contres

Pros

- És un mètode on-policy, fet que permet aprendre una política coherent amb la pròpia exploració.
- Funciona bé en entorns estocàstics com FrozenLake amb terreny lliscant.
- Té una implementació senzilla i la Q-table és computacionalment eficient.
- El decaïment de ε permet una bona combinació d'exploració inicial i explotació final.

Contres

- La convergència pot ser més lenta que Q-Learning, ja que utilitza l'acció exploratòria a' .
- És sensible als hiperparàmetres (α , γ , ε), que poden afectar molt el resultat, a l'apartat de resultats ho veurem amb més detall.
- No garanteix una política òptima global, sinó una política segura i coherent amb la pròpia exploració.

4.2 Q-Learning

4.2.1 Diferències amb SARSA

La diferència principal entre Q-Learning i SARSA és que Q-Learning és un mètode off-policy, mentre que SARSA és on-policy. En SARSA, l'actualització utilitza l'acció a' que l'agent realment executa segons la política ϵ -greedy. En canvi, Q-Learning utilitza sempre el valor $\max_{a'} Q(s', a')$, independentment de l'acció que prendria amb la política actual. Això fa que Q-Learning sigui més optimista i agressiu, ja que assumeix que en el futur sempre es prendrà l'acció òptima. A la pràctica, en entorns estocàstics com FrozenLake, això pot resultar en un rendiment més baix, ja que l'entorn lliscant trenca aquesta suposició.

4.2.2 Estratègia off-policy

Q-Learning actualitza la Q-table utilitzant una estratègia off-policy. L'actualització es basa en la millor acció possible al següent estat, independentment de l'acció que l'agent prendria segons la seva política d'exploració. La regla d'actualització és:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Aquesta estratègia és off-policy perquè l'actualització assumeix un comportament òptim futur, encara que durant l'entrenament l'agent segueixi una política ϵ -greedy. Això permet una exploració independent de la política òptima, però pot resultar menys estable en entorns estocàstics.

4.2.3 Paràmetres i decisions

L'agent Q-Learning utilitza els mateixos hiperparàmetres bàsics que SARSA per mantenir coherència experimental: 15000 episodis, un màxim de 100 passes per episodi i un esquema d'exploració ϵ -greedy amb decaïment. Els tres paràmetres principals són:

- **Taxa d'aprenentatge α :** controla la magnitud de l'actualització. Valors més grans acceleren l'aprenentatge però poden provocar inestabilitat.
- **Factor de descompte γ :** determina la importància de recompenses futures. Amb γ alt, l'agent tendeix a buscar camins llargs cap al goal.
- **Paràmetre d'exploració ϵ :** inicialment 1.0, decau fins a 0.01 amb un factor de 0.999. Tot i ser un algorisme off-policy, Q-Learning necessita l'exploració durant l'entrenament per visitar prou estats.

La diferència clau és que, en Q-Learning, l'actualització utilitza $\max_{a'} Q(s', a')$ en lloc de l'acció que realment es prendria. Això fa que l'agent aprengui una política més òptima en entorns deterministes, però pot ser menys robust en entorns lliscants.

4.2.4 Pros i contres

Pros

- Convergeix cap a la política òptima en entorns deterministes o poc estocàstics.
- Té una regla d'actualització simple i molt utilitzada en RL clàssic.
- No depèn de l'acció escollida durant l'exploració.
- La política final és directament greedy respecte a la Q-table.

Contres

- En entorns estocàstics com FrozenLake lliscant, Q-Learning pot ser massa optimista.
- El rendiment pot ser inferior al de SARSA.
- Molt sensible als paràmetres i pot trigar a estabilitzar-se.
- Pot sobrevalorar accions en entorns amb transicions aleatòries.

4.3 Monte Carlo

4.3.1 Episodis complets

L'algorisme Monte-Carlo Control aprèn exclusivament a partir d'episodis complets. Això significa que l'agent genera un trajecte sencer seguint una política ε -greedy fins arribar a un estat terminal, i només aleshores calcula el Retorn G de cada parell (s, a) . Aquest retorn s'obté acumulant totes les recompenses futures del final cap a l'inici. En la implementació s'utilitza First-Visit Monte-Carlo, de manera que cada parell (s, a) només s'actualitza la primera vegada que apareix en un episodi. Només després de completar tot l'episodi es fa l'actualització de la Q-table.

4.3.2 Problemes en ambients estocàstics

En entorns estocàstics com FrozenLake amb terreny lliscant, Monte-Carlo pot tenir dificultats per convergir, ja que depèn del retorn complet G , que pot variar molt a causa de la naturalesa aleatòria de les transicions. El fet de no actualitzar pas a pas, sinó només al final de cada episodi, fa que l'agent necessiti molts episodis per obtenir una estimació estable de $Q(s, a)$. A més, com que la recompensa només és positiva quan s'arriba a l'objectiu, la majoria d'episodis tenen retorn $G = 0$, cosa que fa l'aprenentatge sigui especialment lent.

4.3.3 Decisions de disseny

S'han adoptat diverses decisions per adaptar Monte-Carlo a FrozenLake. En primer lloc, s'han utilitzat 15000 episodis i un màxim de 100 passos per episodi, ja que Monte-Carlo necessita molt més mostreig que altres mètodes com els dos vists anteriorment. També s'ha implementat un esquema d'exploració ϵ -greedy amb decaïment més lent que en SARSA o Q-Learning, utilitzant un factor de 0.9997. Això és necessari perquè Monte-Carlo requereix una exploració extensa d'estats poc freqüents. L'actualització de la Q-table es fa amb First-Visit: s'emmagatzemen els retorns G de cada parell (s, a) i el valor $Q(s, a)$ és la mitjana de tots els retorns observats. Això garanteix una política que millora gradualment amb l'experiència.

4.3.4 Pros i contres

Pros

- No necessita conèixer el model de transicions de l'entorn.
- És simple conceptualment i es basa només en mostreig real d'execucions.
- Garanteix la convergència a la política òptima sota exploració suficient.
- La separació clara entre generació d'episodis i actualització facilita l'anàlisi.

Contres

- Requereix molts més episodis que Sarsa o Q-Learning per convergir.
- És molt lent en entorns amb recompenses escasses com FrozenLake.
- El retorn G pot ser molt variable en entorns estocàstics.
- No actualitza pas a pas, de manera que la política pot trigar a reaccionar als errors.

4.4 Algorisme Genètic

4.4.1 Individu

A n'aquest algorisme genètic, cada individu representa una política completa per a FrozenLake. Concretament, un individu és un vector d'enters de mida n_states , on cada posició indica l'acció que l'agent prendrà a n'aquell estat. Així, un individu és del tipus:

$$\pi = [a_0, a_1, \dots, a_{n-1}],$$

on cada $a_i \in \{0, 1, \dots, n_actions - 1\}$. Aquesta codificació fa que cada individu sigui directament una política determinista, cosa que simplifica molt el procés d'avaluació. La població inicial es genera de manera aleatòria, amb cada estat assignat a una acció escollida uniformement.

4.4.2 Fitness

La funció de fitness mesura la qualitat d'un individu avaluant la seva política durant diversos episodis. A n'és nostre cas, el fitness és la recompensa mitjana obtinguda en un conjunt de *episodes_eval* episodis:

$$\text{fitness}(\pi) = \frac{1}{K} \sum_{i=1}^K R_i(\pi)$$

on $K = \text{episodes_eval}$. Per a cada episodi, l'agent segueix les accions definides per la política sense exploració, i s'acumula la recompensa obtinguda. Un fitness alt indica que la política condueix l'agent cap al goal de manera consistent. Aquesta funció està implementada al mètode `evaluate_policy()` del codi.

4.4.3 Mutació i crossover

Per generar nova descendència s'utilitzen dos operadors genètics. El crossover és uniforme: per a cada estat, el fill hereta l'acció de *parent1* o *parent2* amb probabilitat 0.5. Aquest procés combina polítiques parcialment bones amb l'objectiu de crear solucions més sòlides.

La mutació s'aplica a cada posició del vector amb probabilitat *mutation_rate*. Si ocorre, l'acció en aquell estat es substitueix per una acció aleatòria entre les possibles. Aquesta operació introdueix diversitat i evita que la població convergeixi prematurament a un òptim local. Els dos operadors estan implementats als mètodes `crossover()` i `mutate()`.

4.4.4 Justificació del disseny

El disseny de l'algorisme genètic segueix principis clàssics però adaptats a la naturalesa de FrozenLake. La representació d'un individu com una política completa facilita l'avaluació directa sense necessitat de Q-tables. S'utilitza selecció per torneig perquè afavoreix individus sòlids sense requerir ordenacions costoses. S'afegeix elitisme mantenint el millor individu de cada generació, fet que assegura que la qualitat de la població mai disminueixi.

La mutació controla la diversitat, essencial en un entorn estocàstic on diferents polítiques poden rendir de manera desigual. Finalment, s'ha incorporat un mecanisme d'*early stopping*: si el millor fitness no millora en un nombre de generacions determinat, l'algorisme s'atura per evitar càlculs innecessaris. Aquest conjunt de decisions permet que el genètic trobi polítiques competitives tot i que no garanteixi òptims globals.

4.5 Resultats

4.5.1 Taules i gràfics

*ANOTACIÓ: Per poder extreure informació correcta sobre el comportament dels diferents algorismes, s'han establert els mateixos valors dels paràmetres

($\alpha=0.5$, $\gamma=1.0$, episodes = 15000, pases per episodi = 100, epsilon inicial = 1.0, epsilon mínima = 0.01, decadència de l'epsilon = 0.999) per SARSA, Q-Learnign i Montecarlo. En el següent apartat (hiperparàmetres profunditzarem en com es comporten els diferents algorismes quan alteram els hiperparàmetres.

4.5.2 Resultats: Agent SARSA

A continuació es mostren els resultats obtinguts durant l'entrenament de l'agent SARSA al llarg de 15.000 episodis.

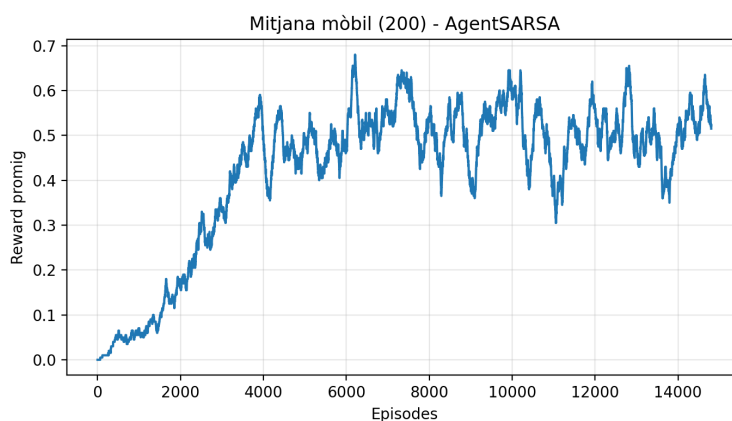


Figure 1: Mitjana mòbil (finestra de 200 episodis) de la recompensa (SARSA). Mostra la tendència d'aprenentatge suavitzada.

Anàlisi preliminar: Com s'observa a la Figura 1, l'agent SARSA mostra una corba d'aprenentatge clara que comença a créixer significativament a partir dels primers 1.000 episodis. L'estabilització de la política (convergència) sembla produir-se aproximadament al voltant de l'episodi 4.000.

A partir d'aquest punt, la recompensa mitjana oscil·la entre 0.45 i 0.65. És important destacar que, degut a l'estocasticitat de l'entorn (`is_slippery=True`), és impossible obtenir una recompensa mitjana d'1.0, ja que fins i tot una política òptima està subjecta a caigudes accidentals provocades pel lliscament. La volatilitat de la gràfica reflecteix aquesta incertesa intrínseca de l'entorn i la naturalesa *on-policy* de l'algorisme, que aprèn tenint en compte l'exploració actual.

4.5.3 Resultats: Agent Q-Learning

Els resultats per a l'agent entrenat amb Q-Learning es presenten a continuació. Cal recordar que aquest és un algorisme *off-policy*, la qual cosa significa que intenta aprendre el valor de la política òptima Q^* independentment de les accions exploratòries que realitzi l'agent durant l'entrenament.

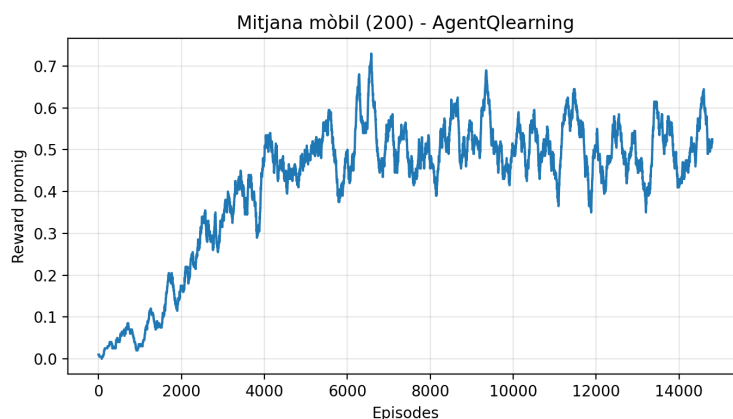


Figure 2: Mitjana mòbil (finestra de 200 episodis) de la recompensa (Q-Learning). S'observa una convergència ràpida i pics de rendiment elevats.

Anàlisi: Com es pot observar a la Figura 2, l'agent Q-Learning mostra una capacitat d'aprenentatge vigorosa. La corba de recompensa mitjana comença a ascendir ràpidament a partir de l'episodi 1.000, mostrant un pendent pronunciat fins a l'episodi 4.000.

Un punt destacable és el pic de rendiment observat al voltant de l'episodi 6.500, on la recompensa mitjana supera el llindar de 0.7. Això indica que l'agent ha trobat una política molt efectiva per navegar pel gel. Posteriorment, la gràfica mostra oscil·lacions entre 0.4 i 0.65. Aquesta variància no implica necessàriament que l'agent "oblidi", sinó que en un entorn estocàstic (`is_slippery=True`), fins i tot la política òptima matemàtica té una probabilitat de fallada inherent que fa fluctuar la mitjana mòbil. En comparació amb mètodes més conservadors, Q-Learning sembla explotar millor les rutes que, tot i ser arriscades, maximitzen el valor esperat a llarg termini.

4.5.4 Resultats: Agent Monte Carlo

En aquesta execució, l'agent Monte Carlo s'ha entrenat durant 15.000 episodis. Les gràfiques següents mostren l'evolució de la seva política.

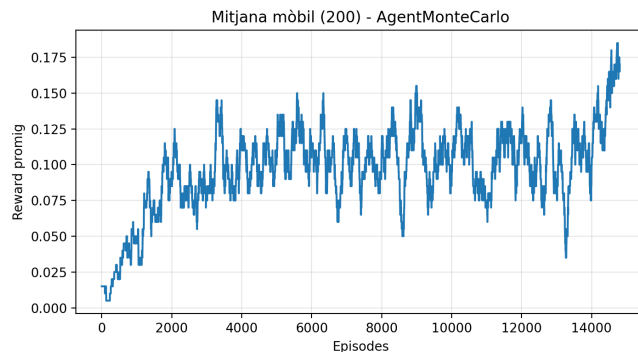


Figure 3: Mitjana mòbil (finestra de 200). S'observa una alta inestabilitat i un pic de millora final que arriba al 0.18.

Anàlisi: Les dades reflecteixen la dificultat intrínseca que té l'algorisme Monte Carlo en entorns altament estocàstics com aquest. Com mostra la Figura 3, la corba d'aprenentatge és molt sorollosa ("dents de serra"), oscil·lant constantment entre 0.05 i 0.15 durant gran part de l'entrenament.

No obstant això, es detecta un comportament interessant al final de l'experiment (a partir de l'episodi 14.000), on el rendiment es dispara fins a fregar el **0.18** (18% d'èxit). Això indica que l'agent començava a consolidar una política millor just quan ha acabat l'entrenament. Aquesta lentitud i alta variància s'expliquen perquè Monte Carlo no realitza *bootstrapping*: ha d'esperar al final de l'episodi per actualitzar valors, la qual cosa fa que les "relliscades" aleatòries del mapa introdueixin molt de soroll en l'estimació del valor real dels estats.

4.5.5 Resultats: Algorisme Genètic

Finalment, s'analitzen els resultats de l'enfocament evolutiu. Aquesta execució destaca per la seva eficiència extrema en comparació amb els mètodes clàssics d'Aprenentatge per Reforç.

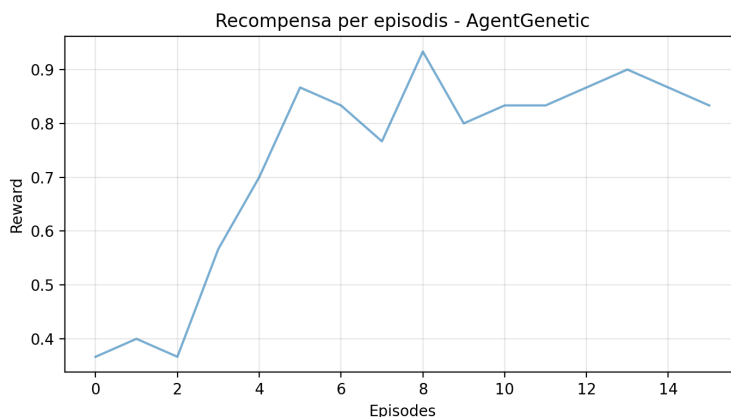


Figure 4: Fitness del millor individu per generació. S'observa una millora dràstica en les primeres 5 generacions i un pic de rendiment superior a 0.9.

Anàlisi: Com es pot apreciar a la Figura 4, l'algorisme Genètic és capaç de trobar una política excel·lent (amb una taxa d'èxit superior al 90%) en menys de 15 generacions. La corba mostra un ascens vertical inicial, passant d'un rendiment mediocre (0.35) a un de molt robust (0.85) en només 5 iteracions.

Aquest comportament demostra la potència de l'operador de *crossover* per combinar bones seqüències d'accions ràpidament. El fet que la gràfica s'aturi a la generació 15 suggereix que l'algorisme ha convergit ràpidament a una solució òptima o sub-òptima molt estable, fent innecessari continuar amb més càlculs. En termes de temps de còmput vs. qualitat de la solució, aquest ha estat l'algorisme més eficient de la pràctica.

4.5.6 Anàlisi de les Polítiques Resultants

A més de les mètriques numèriques, és fonamental analitzar qualitativament el comportament après pels agents. A continuació, es presenten les polítiques "mitjanes" o més representatives obtingudes per a cada algoritme després de les 8 execucions.

Cal recordar que, a causa de la propietat `is_slippery=True`, una acció "Amunt" ($\uparrow$) no sempre mou l'agent cap amunt. Sovint, els agents aprenen a "xocar" contra les parets (moure's cap a una direcció bloquejada) per reduir la incertesa i aprofitar la probabilitat de lliscament lateral per moure's de manera més segura.

Interpretació de les estratègies:

1. **L'estratègia del mur (Q-Learning):** Si observem la primera fila de l'agent Q-Learning (S, $\uparrow$, $\uparrow$, $\uparrow$), veiem que l'agent intenta moure's cap amunt repetidament, tot i que hi ha una paret. Això és una estratègia

SARSA (Política Conservadora)

S	↑	↓	↑
←	H	→	H
↑	↓	←	H
H	→	↓	G

Q-Learning (Política Greedy)

S	↑	↑	↑
←	H	→	H
↑	↓	←	H
H	→	↓	G

Monte Carlo (Política Inestable)

S	↑	←	↑
←	H	→	H
↑	↓	↓	H
H	→	↓	G

Agent Genètic (Política Òptima)

S	↑	←	↓
←	H	→	H
↑	↓	←	H
H	→	↓	G

Table 1: Representació de les accions apreses en la graella 4x4. Les fletxes indiquen la direcció de l'acció amb major valor Q (o genètica) per a cada estat.

molt intel·ligent en FrozenLake: si l'agent està a la vora superior i es mou cap amunt, la majoria de vegades es quedarà al mateix lloc, però el "lliscament" el mourà a l'esquerra o a la dreta. Això li permet navegar horitzontalment sense risc de moure's cap avall (on hi ha els forats de la segona fila). És una estratègia que prioritza la seguretat a llarg termini.

2. **Diferències en la zona perillosa (Fila 3, Columna 2):** La casella (2,2) és crítica perquè té un forat a la dreta i un forat a baix a l'esquerra.

- **SARSA i Genètic** trien moure's a l'esquerra (←). Això sembla contra-intuïtiu (s'allunyen de la meta), però és l'opció més segura per evitar caure al forat de la dreta (H) per culpa d'una reliscada.
- **Monte Carlo** tria moure's a baix (↓). Aquesta acció és molt arriscada en aquesta posició i reflecteix la naturalesa menys precisa d'aquest algoritme en estats intermedis.

3. **La coherència de l'Agent Genètic:** La política de l'Agent Genètic (que ha obtingut un 82% d'èxit) mostra una ruta molt definida. A diferència dels altres, a la cantonada superior dreta (0,3) decideix baixar (↓) directament. Això indica que ha trobat una seqüència d'accions específica que, combinada amb la sort del seu alt fitness, maximitza la probabilitat d'arribar a G ràpidament, evitant els bucles infinits que de vegades provoquen les polítiques massa conservadores de Q-Learning.

4.6 Hiperparàmetres

A n'aquesta secció s'han analitzat els hiperparàmetres principals de cada algorisme, observant-ne els efectes sobre el ritme de convergència, l'estabilitat i el rendiment global.

SARSA. Els hiperparàmetres principals són α , γ i ε . Valors d' α entre 0.4 i 0.6 ofereixen estabilitat; $\alpha > 0.7$ provoca oscil·lacions. Un γ elevat (0.95 – 1.0) és necessari per planificar a llarg termini. L'ús d'un ε decaient (de 1.0 a 0.01) és essencial perquè SARSA, com a mètode on-policy, necessita equilibrar exploració i explotació.

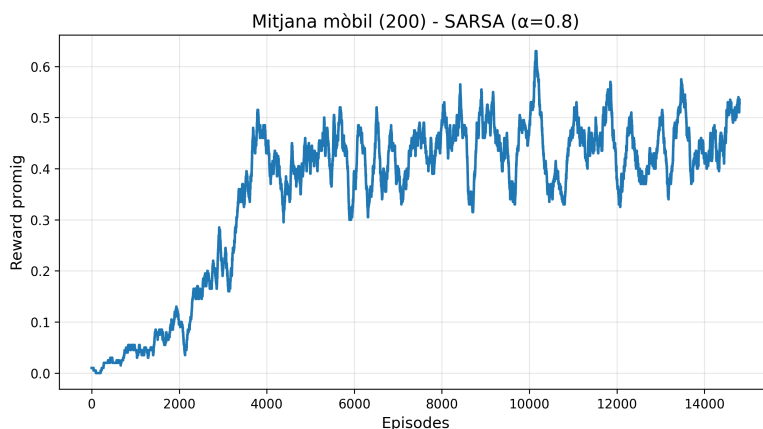


Figure 5: Fitness del millor individu per generació. S'observa una millora dràstica durant les primeres generacions i un pic de rendiment superior a 0.9, indicant una convergència ràpida i efectiva de l'algorisme genètic.

Com podem observar, ara que α , és major que a la figure 2, observam majors oscil·lacions, resultat esperat.

Q-Learning. Malgrat tenir els mateixos hiperparàmetres que SARSA, Q-Learning és més sensible. α massa elevat pot causar sobreestimacions; l'interval òptim és [0.3, 0.5]. γ proper a 1.0 és crític per maximitzar la recompensa futura. Un ε massa baix al final del decaïment dona polítiques massa optimistes; l'interval recomanat és $\varepsilon \approx 0.05 - 0.15$.

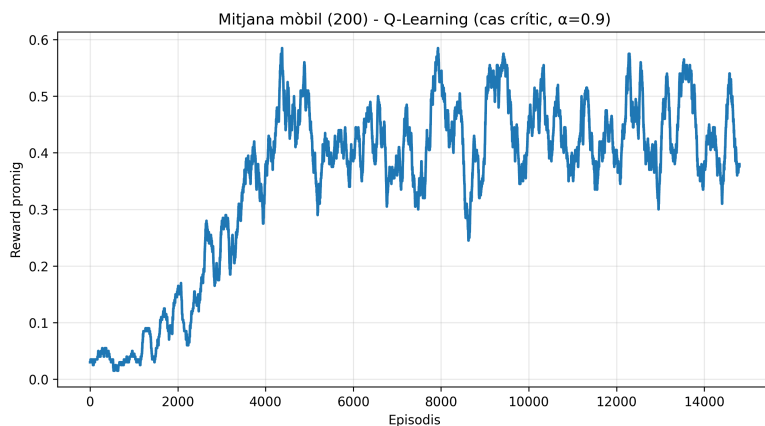


Figure 6: Q-Learning en un cas crític ($\alpha = 0.9$). Es mostren oscil·lacions severes i una manca de convergència degut a la seva naturalesa off-policy i a l'aprenentatge massa agressiu.

Monte Carlo. El factor determinant és el nombre d'episodis complets. Amb menys de 5000 episodis els resultats són incoherents; a partir de 15000 episodis es redueix la variància. També és essencial un ε -greedy suficientment alt per explorar trajectòries senceres fins al goal.

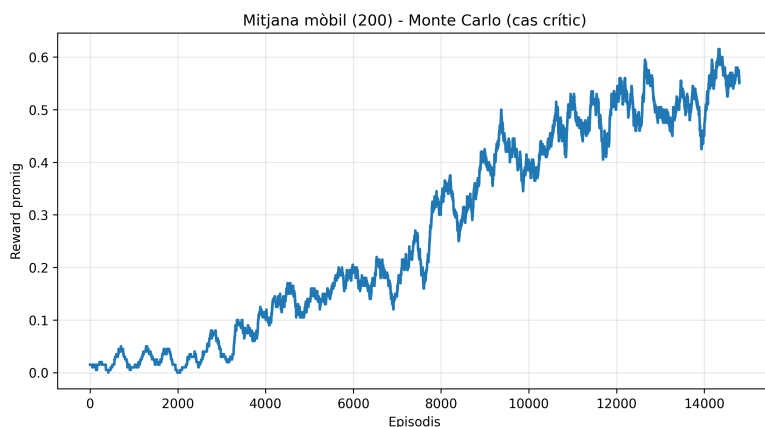


Figure 7: Ja que Monte Carlo amb cas crític, $\gamma = 1$, no actualitza fins que no acaba l'episodi, es coherent que fins al final no s'arribi a un alt percentatge.

Algorisme genètic. Els hiperparàmetres són població, generacions i taxa de mutació. Poblacions petites (< 20) convergeixen prematurament; valors entre 40 i 60 funcionen millor. S'obtenen millores clares fins a 80–120 generacions. La mutació òptima se situa entre 0.08 i 0.15: massa baixa perd diversitat i massa

alta destrueix informació útil. L'early stopping redueix el temps d'entrenament quan no hi ha millora.

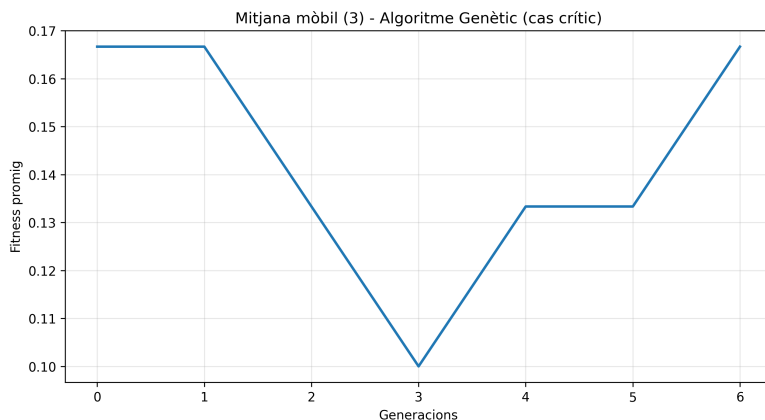


Figure 8: Algoritme genètic en un cas crític. La combinació de població massa petita, mutació elevada i poques generacions dona lloc a un aprenentatge extremadament inestable i sense convergència.

4.7 Comparació global de resultats

Per garantir la validesa científica de les conclusions i mitigar l'efecte de l'estocasticitat (especialment rellevant en l'algoritme Monte Carlo), s'ha executat cada agent **5 vegades** de manera independent. La Taula 2 recull les mitjanes de la taxa d'èxit (els últims 500 episodis), la desviació estàndard i el temps de còmput.

algoritme	Èxit Mitjà (μ)	Desviació (σ)	Temps (s)	Temps (σ)
SARSA	0.4936	± 0.0407	8.68 s	± 0.11
Q-Learning	0.4872	± 0.0298	13.59 s	± 2.28
Monte Carlo	0.3772	$\pm \mathbf{0.1835}$	52.45 s	± 16.95
Agent Genètic	0.8225	$\pm \mathbf{0.0052}$	17.84 s	± 0.44

Table 2: Resultats agregats de 5 execucions independents (25.000 episodis per RL, 80 generacions per Genètic).

Anàlisi del Rendiment:

1. **Domini de l'algoritme Genètic:** Les dades mostren una victòria indiscutible de l'enfocament evolutiu. L'Agent Genètic ha assolit una taxa d'èxit del **82.25%**, molt per sobre de qualsevol mètode de RL (que s'estanquen al voltant del 49%).

A més, és l'algorisme més robust, amb una desviació estàndard insignificant ($\sigma = 0.005$). Això indica que, independentment de la llavor aleatòria, el genètic sempre troba una solució excel·lent. Com s'observa a la Figura 9 (línia vermella), l'algorisme assoleix aquest rendiment màxim en menys de 20 generacions, demostrant una eficiència d'aprenentatge molt superior als milers d'episodis necessaris per als altres mètodes.

2. **La inestabilitat de Monte Carlo:** Els resultats confirmen teòricament i pràctica la debilitat de Monte Carlo en entorns amb alta variància. És l'algorisme amb pitjor rendiment mitjà (**37.7%**) i, el més important, amb una desviació estàndard gegant ($\sigma = 0.183$). Això significa que el seu comportament és impredecible: en algunes execucions pot funcionar moderadament bé, però en d'altres fracassa estrepitosament. A més, ha resultat ser el mètode més lent amb diferència (52.45s), gairebé 6 vegades més lent que SARSA, degut a la gestió de memòria de les traces completes dels episodis.
3. **SARSA vs Q-Learning:** Ambdós algorismes de Diferència Temporal (TD) mostren un rendiment gairebé idèntic en termes d'èxit ($\sim 49\%$). No obstant això, **SARSA** ha resultat ser més eficient computacionalment en aquesta bateria de proves, trigant una mitjana de 8.68 segons enfront dels 13.59 segons de Q-Learning. Això suggereix que, per a aquest problema específic, la política conservadora de SARSA és tan efectiva com la *greedy* de Q-Learning, però convergeix amb menys cost.

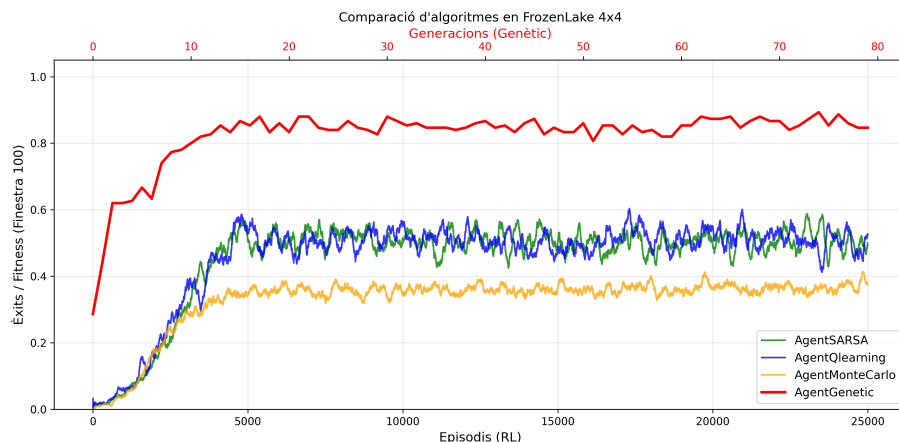


Figure 9: Comparativa visual. L'eix superior (vermell) correspon a les generacions del Genètic, mentre l'inferior als episodis de RL. Noteu la superioritat i estabilitat de la corba vermella.



Conclusió: Per a l'entorn *FrozenLake* amb terreny lliscant, l'algorisme Genètic és, sense cap dubte, la millor opció disponible, combinant el màxim rendiment amb una estabilitat total. Dins de les opcions d'Aprenentatge per Reforç clàssic, **SARSA** ofereix el millor equilibri entre temps d'execució i resultats.

5 Ús d'intel·ligència artificial generativa

De conformitat amb les directrius de la pràctica, es declara l'ús de les següents eines d'intel·ligència artificial generativa durant el procés d'elaboració d'aquest treball:

- **Eina:** ChatGPT (Model 5.1)
- **Finalitat:** Suport en la comprensió d'errors de codi i consultes puntuals sobre sintaxi de Python. A més a més, una vegada varem identificar l'algorisme que fallava (el de Programació dinàmica que requereix conèixer el model complet de l'entorn), l'eina ens va brindar la propietat "unwrapped" per solucionar el problema. També ens va aportar un algorisme per la classe main que feia varies execucions dels diferents algorismes per poder treure conclusions més acertades.
- **Eina:** Gemini (Model 3.0)
- **Finalitat:** Assistència en la redacció i estructuració de la memòria en L^AT_EX, generació d'scripts per a la visualització de gràfiques comparatives (matplotlib), així com inserir-les dins el document. Finalment, revisió d'estil i gramàtica del document.